PS1 - IO

Tate Mason

```
# Packages
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(lubridate)
library(stargazer)
```

```
Please cite as:

 Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.
 R package version 5.2.3. https://CRAN.R-project.org/package=stargazer
```

```
library(fixest)
library(here)
```

```
here() starts at /Users/tate/SchoolWork
```

```r
library(readr)
library(tidyr)
library(stringr)


# Data
iri <- read_tsv("~/SchoolWork/Y2S1/IO/PS1/IRI.csv", show_col_types = FALSE)
names(iri) <- gsub("^t", "", names(iri))  # Clean names
iri <- iri %>%
  mutate(
    market_id = interaction(store_id, week_id, drop = TRUE),
    date_id = interaction(year, month, drop = TRUE),
    date = as.Date(paste(year, month, "01", sep = "-"))
  )
```

Question 1:

1.1

```r
# Distinct counts per market
brands_per_market <- iri %>%
  distinct(market_id, brand) %>%
  count(market_id, name = "n_brands")

mfrs_per_market <- iri %>%
  distinct(market_id, parent) %>%
  count(market_id, name = "n_mfrs")

# Total market sales per market
total_sales_market <- iri %>%
  group_by(market_id) %>%
  summarise(total_sales = sum(quantity, na.rm = TRUE), .groups = "drop")

# Brand-level sales (market x brand)
brand_sales_market <- iri %>%
  group_by(market_id, brand) %>%
  summarise(brand_sales = sum(quantity, na.rm = TRUE), .groups = "drop")

# Per-market price & characteristics (means within market)
```

```r
market_chars <- iri %>%
  group_by(market_id) %>%
  summarise(
    price_mean     = mean(price, na.rm = TRUE),
    fiber_mean     = mean(fiber, na.rm = TRUE),
    sugars_mean    = mean(sugars, na.rm = TRUE),
    flavored_mean  = mean(as.numeric(flavored), na.rm = TRUE),
    fortified_mean = mean(as.numeric(fortified), na.rm = TRUE),
    .groups = "drop"
  )

# Helper to summarise numeric vectors
summarise_vec <- function(x) {
  tibble(
    mean   = mean(x, na.rm = TRUE),
    sd     = sd(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    max    = max(x, na.rm = TRUE)
  )
}

# Tables for 1.1
tbl_counts <- bind_cols(
  variable = c("n_brands", "n_mfrs"),
  bind_rows(
    summarise_vec(brands_per_market$n_brands),
    summarise_vec(mfrs_per_market$n_mfrs)
  )
)

tbl_total_sales <- bind_cols(
  variable = "total_sales (per market)",
  summarise_vec(total_sales_market$total_sales)
)

tbl_brand_sales <- bind_cols(
  variable = "brand_sales (per market-brand)",
  summarise_vec(brand_sales_market$brand_sales)
)

tbl_price_chars <- tribble(
  ~variable,                ~mean, ~sd, ~median, ~max,
```

```
  "price (market mean)",
    summarise_vec(market_chars$price_mean)$mean,
    summarise_vec(market_chars$price_mean)$sd,
    summarise_vec(market_chars$price_mean)$median,
    summarise_vec(market_chars$price_mean)$max,
  "fiber (market mean)",
    summarise_vec(market_chars$fiber_mean)$mean,
    summarise_vec(market_chars$fiber_mean)$sd,
    summarise_vec(market_chars$fiber_mean)$median,
    summarise_vec(market_chars$fiber_mean)$max,
  "sugars (market mean)",
    summarise_vec(market_chars$sugars_mean)$mean,
    summarise_vec(market_chars$sugars_mean)$sd,
    summarise_vec(market_chars$sugars_mean)$median,
    summarise_vec(market_chars$sugars_mean)$max,
  "flavored share (market mean)",
    summarise_vec(market_chars$flavored_mean)$mean,
    summarise_vec(market_chars$flavored_mean)$sd,
    summarise_vec(market_chars$flavored_mean)$median,
    summarise_vec(market_chars$flavored_mean)$max,
  "fortified share (market mean)",
    summarise_vec(market_chars$fortified_mean)$mean,
    summarise_vec(market_chars$fortified_mean)$sd,
    summarise_vec(market_chars$fortified_mean)$median,
    summarise_vec(market_chars$fortified_mean)$max
)

summary_11 <- bind_rows(
  tbl_counts,
  tbl_total_sales,
  tbl_brand_sales,
  tbl_price_chars
)

summary_11
```

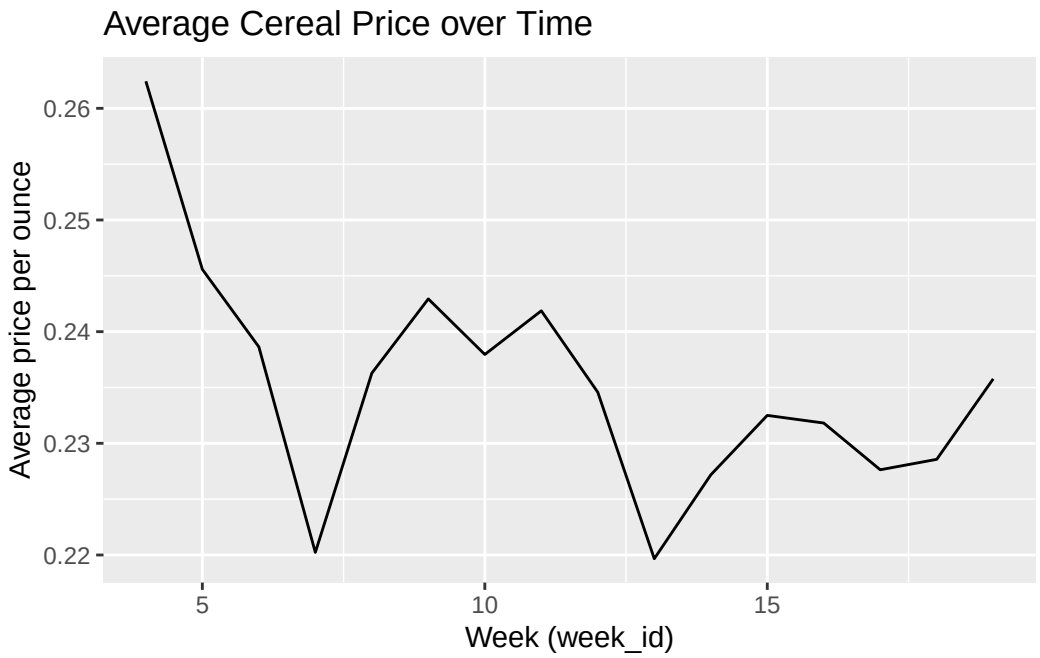
```
# A tibble: 9 x 5
  variable                          mean         sd    median       max
  <chr>                            <dbl>      <dbl>     <dbl>     <dbl>
1 n_brands                        34.9       4.82      37        39
2 n_mfrs                           3.95      0.259      4         4
3 total_sales (per market)     15555.    12358.     12557.    84567.
```

```
4 brand_sales (per market-brand)   446.      726.       238.    26470.
5 price (market mean)                0.235     0.0265     0.233     0.325
6 fiber (market mean)                1.30      0.0874     1.31      1.67
7 sugars (market mean)               8.31      0.205      8.33     10.2
8 flavored share (market mean)       0.640     0.0265     0.641     0.857
9 fortified share (market mean)      0.485     0.0341     0.487     0.667
```

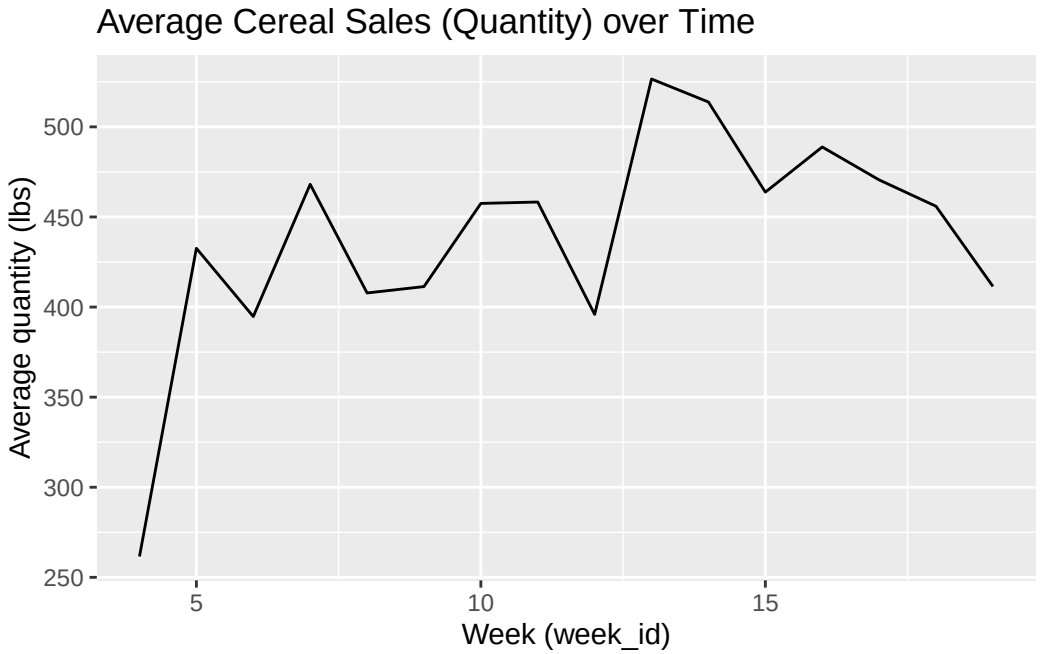
```
# If you want a file:
# write_csv(summary_11, "summary_11_descriptives.csv")
```

1.2

```
ts_simple <- iri %>%
  group_by(week_id) %>%
  summarise(
    avg_price = mean(price, na.rm = TRUE),
    avg_qty   = mean(quantity, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(week_id)

p_price <- ggplot(ts_simple, aes(x = week_id, y = avg_price)) +
  geom_line() +
  labs(title = "Average Cereal Price over Time",
       x = "Week (week_id)", y = "Average price per ounce")

p_qty <- ggplot(ts_simple, aes(x = week_id, y = avg_qty)) +
  geom_line() +
  labs(title = "Average Cereal Sales (Quantity) over Time",
       x = "Week (week_id)", y = "Average quantity (lbs)")

p_price
```

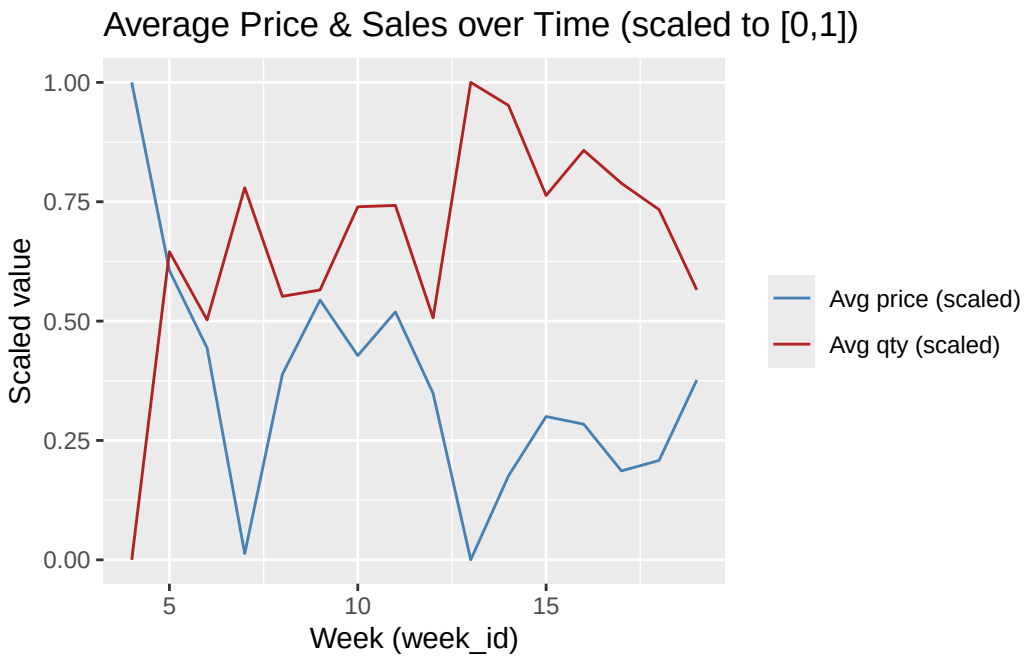



```
p_qty
```




```
# Save figures
ggsave("q12-ts-1.pdf", p_price, width = 10, height = 6, dpi = 300)
```

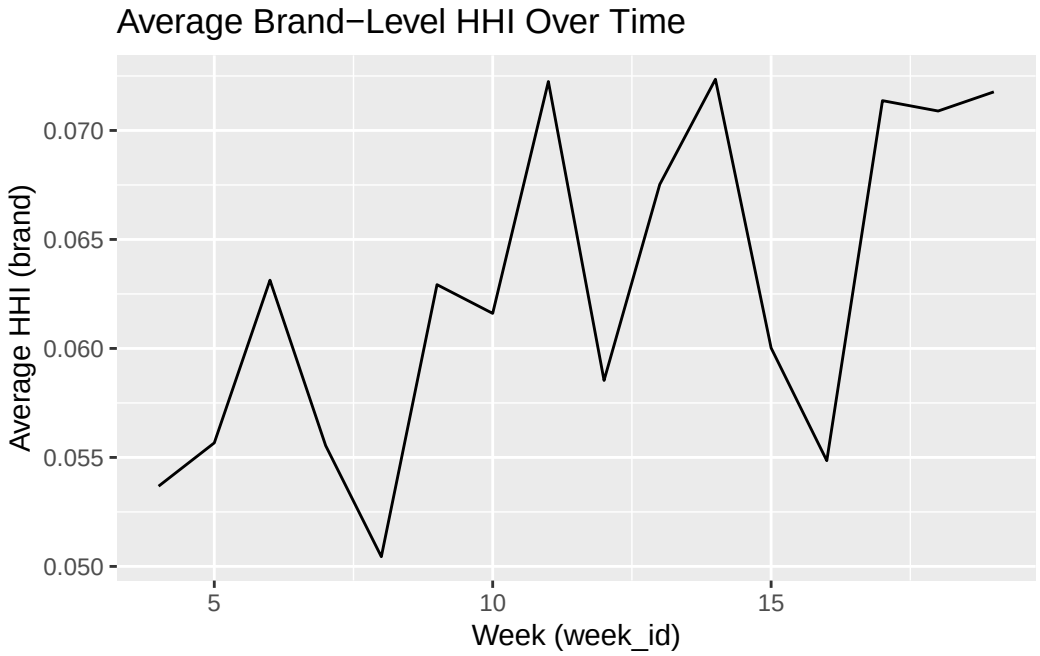
```r
ggsave("q12-ts-2.pdf", p_qty, width = 10, height = 6, dpi = 300)

# Dual-axis style (scaled)
ts_long_scaled <- ts_simple %>%
  mutate(
    price_scaled = (avg_price - min(avg_price, na.rm = TRUE)) /
                   (max(avg_price, na.rm = TRUE) - min(avg_price, na.rm = TRUE)),
    qty_scaled   = (avg_qty - min(avg_qty, na.rm = TRUE)) /
                   (max(avg_qty, na.rm = TRUE) - min(avg_qty, na.rm = TRUE))
  ) %>%
  select(week_id, price_scaled, qty_scaled) %>%
  pivot_longer(-week_id, names_to = "series", values_to = "value")

p_dual <- ggplot(ts_long_scaled, aes(x = week_id, y = value, color = series)) +
  geom_line() +
  scale_color_manual(values = c("price_scaled" = "steelblue", "qty_scaled" = "firebrick"),
                     labels = c("Avg price (scaled)", "Avg qty (scaled)")) +
  labs(title = "Average Price & Sales over Time (scaled to [0,1])",
       x = "Week (week_id)", y = "Scaled value", color = NULL)

p_dual
```



```r
ggsave("q12-ts-3.png", p_dual, width = 10, height = 6, dpi = 300)
```

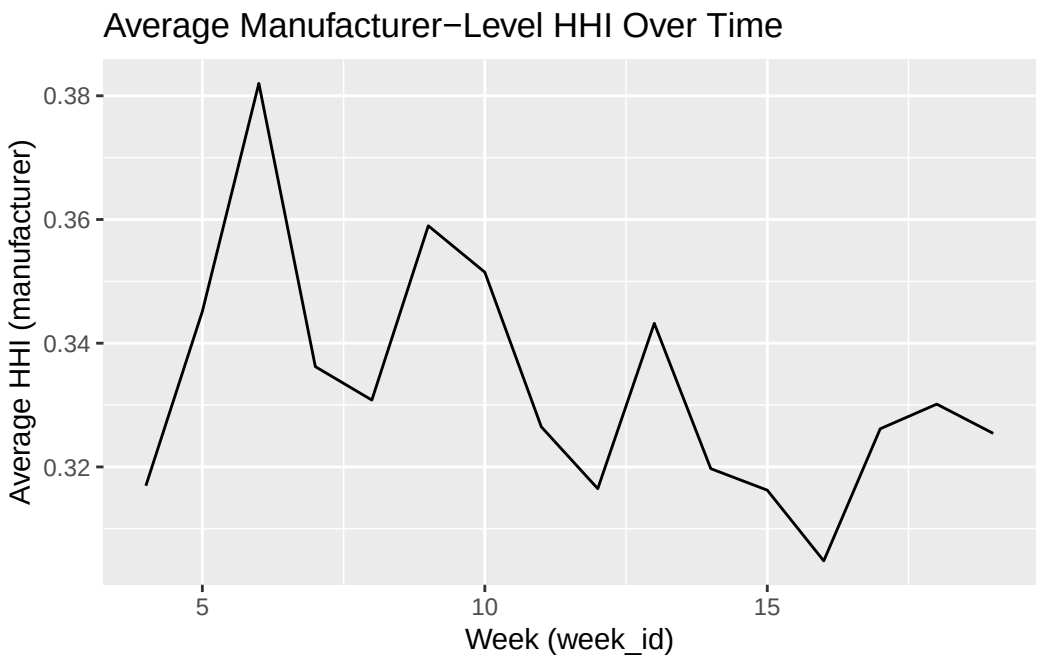
1.3

```r
# Brand HHI per market
brand_shares <- iri %>%
  group_by(market_id) %>%
  mutate(total_mkt_sales = sum(quantity, na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(market_id, brand) %>%
  summarise(brand_sales = sum(quantity, na.rm = TRUE),
            total_mkt_sales = first(total_mkt_sales),
            .groups = "drop") %>%
  mutate(share_brand = if_else(total_mkt_sales > 0, brand_sales / total_mkt_sales, 0))

hhi_brand_market <- brand_shares %>%
  group_by(market_id) %>%
  summarise(HHI_brand = sum(share_brand^2, na.rm = TRUE), .groups = "drop")

hhi_brand_week <- iri %>%
  distinct(market_id, week_id) %>%
  inner_join(hhi_brand_market, by = "market_id") %>%
  group_by(week_id) %>%
  summarise(avg_HHI_brand = mean(HHI_brand, na.rm = TRUE), .groups = "drop") %>%
  arrange(week_id)

p_hhi_brand <- ggplot(hhi_brand_week, aes(x = week_id, y = avg_HHI_brand)) +
  geom_line() +
  labs(title = "Average Brand-Level HHI Over Time",
       x = "Week (week_id)", y = "Average HHI (brand)")

p_hhi_brand
```

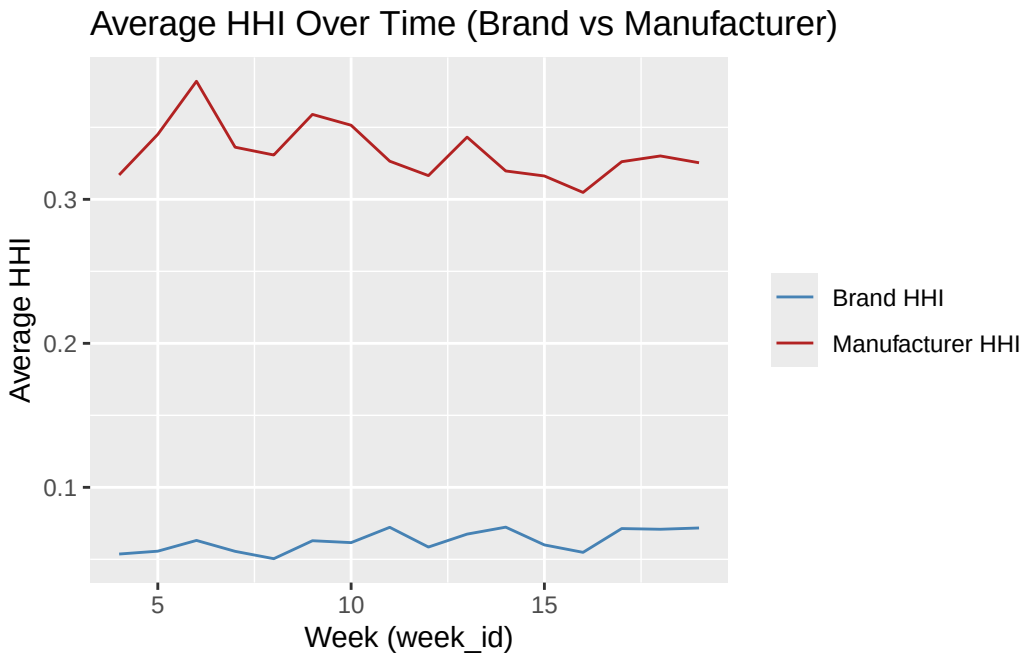



```
ggsave("q13-hhi-1.pdf", p_hhi_brand, width = 10, height = 6, dpi = 300)

# Manufacturer (parent) HHI per market
parent_shares <- iri %>%
  group_by(market_id) %>%
  mutate(total_mkt_sales = sum(quantity, na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(market_id, parent) %>%
  summarise(parent_sales = sum(quantity, na.rm = TRUE),
            total_mkt_sales = first(total_mkt_sales),
            .groups = "drop") %>%
  mutate(share_parent = if_else(total_mkt_sales > 0, parent_sales / total_mkt_sales, 0))

hhi_parent_market <- parent_shares %>%
  group_by(market_id) %>%
  summarise(HHI_parent = sum(share_parent^2, na.rm = TRUE), .groups = "drop")

hhi_parent_week <- iri %>%
  distinct(market_id, week_id) %>%
  inner_join(hhi_parent_market, by = "market_id") %>%
  group_by(week_id) %>%
  summarise(avg_HHI_parent = mean(HHI_parent, na.rm = TRUE), .groups = "drop") %>%
  arrange(week_id)
```

```r
p_hhi_parent <- ggplot(hhi_parent_week, aes(x = week_id, y = avg_HHI_parent)) +
  geom_line() +
  labs(title = "Average Manufacturer-Level HHI Over Time",
       x = "Week (week_id)", y = "Average HHI (manufacturer)")

p_hhi_parent
```




```r
ggsave("q13-hhi-2.pdf", p_hhi_parent, width = 10, height = 6, dpi = 300)

# Combined comparison
hhi_both <- hhi_brand_week %>% inner_join(hhi_parent_week, by = "week_id")

p_hhi_both <- ggplot(hhi_both, aes(x = week_id)) +
  geom_line(aes(y = avg_HHI_brand, color = "Brand HHI")) +
  geom_line(aes(y = avg_HHI_parent, color = "Manufacturer HHI")) +
  scale_color_manual(values = c("Brand HHI" = "steelblue", "Manufacturer HHI" = "firebrick")) +
  labs(title = "Average HHI Over Time (Brand vs Manufacturer)",
       x = "Week (week_id)", y = "Average HHI", color = NULL)

p_hhi_both
```




```
ggsave("q13-hhi-3.pdf", p_hhi_both, width = 10, height = 6, dpi = 300)
```

Question 2:

2.1

```
# Prepare data for regression
reg_data <- iri %>%
  group_by(market_id) %>%
  mutate(
    market_fe = as.factor(store_id),
    manufacturer_fe = as.factor(parent),
    time_fe = as.factor(week_id),
    brand_fe = as.factor(brand),
    p_jmt = price,
    total_sales = sum(quantity, na.rm = TRUE),
    sj = quantity / M,
    s0 = 1 - (total_sales / M),
    u = log(sj) - log(s0)
  ) %>%
  ungroup()
```

Our estimation model is thus:

$$u_{jmt} = \alpha p_{jmt} + \beta \mathbb{X}_{jmt} + \xi_{jmt}$$

where $\mathbb{X}_{jmt}$ includes the product characteristics fiber, sugars, flavored, and fortified, as well as fixed effects for market, manufacturer, and time. and u_{jmt} is the log-odds of product j's market share in market m at time t.

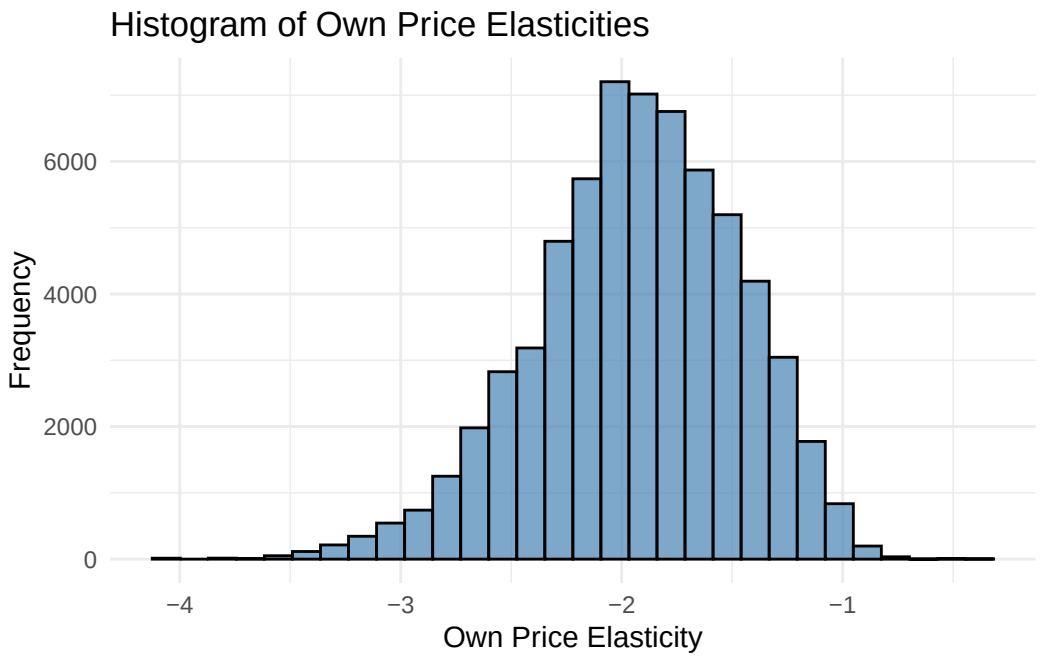
2.2

```
# Regression model
list <- c("fiber", "sugars", "flavored", "fortified",
          "market_fe", "manufacturer_fe", "time_fe")
reg_model <- feols(
  u ~ p_jmt + fiber + sugars + flavored + fortified |
  market_fe + manufacturer_fe + time_fe,
  data = reg_data
)

summary(reg_model)
```

```
OLS estimation, Dep. Var.: u
Observations: 63,966
Fixed-effects: market_fe: 642,  manufacturer_fe: 4,  time_fe: 16
Standard-errors: IID
            Estimate Std. Error   t value  Pr(>|t|)
p_jmt     -8.314431   0.083677 -99.36340 < 2.2e-16 ***
fiber     -0.118828   0.003300 -36.01266 < 2.2e-16 ***
sugars    -0.043445   0.001077 -40.33986 < 2.2e-16 ***
flavored   0.192915   0.008305  23.22877 < 2.2e-16 ***
fortified  0.004322   0.009132   0.47327   0.63602
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 0.899505     Adj. R2: 0.248711
                 Within R2: 0.151216
```

```
# Histogram of own price elasticities
reg_data <- reg_data %>%
  mutate(own_price_elasticity = coef(reg_model)["p_jmt"] * p_jmt * (1 - sj))
```

```r
ggplot(reg_data, aes(x = own_price_elasticity)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black", alpha = 0.7) +
  labs(title = "Histogram of Own Price Elasticities",
       x = "Own Price Elasticity", y = "Frequency") +
  theme_minimal()
```




```r
ggsave("q2-2-1.pdf", width = 10, height = 6, dpi = 300)
```

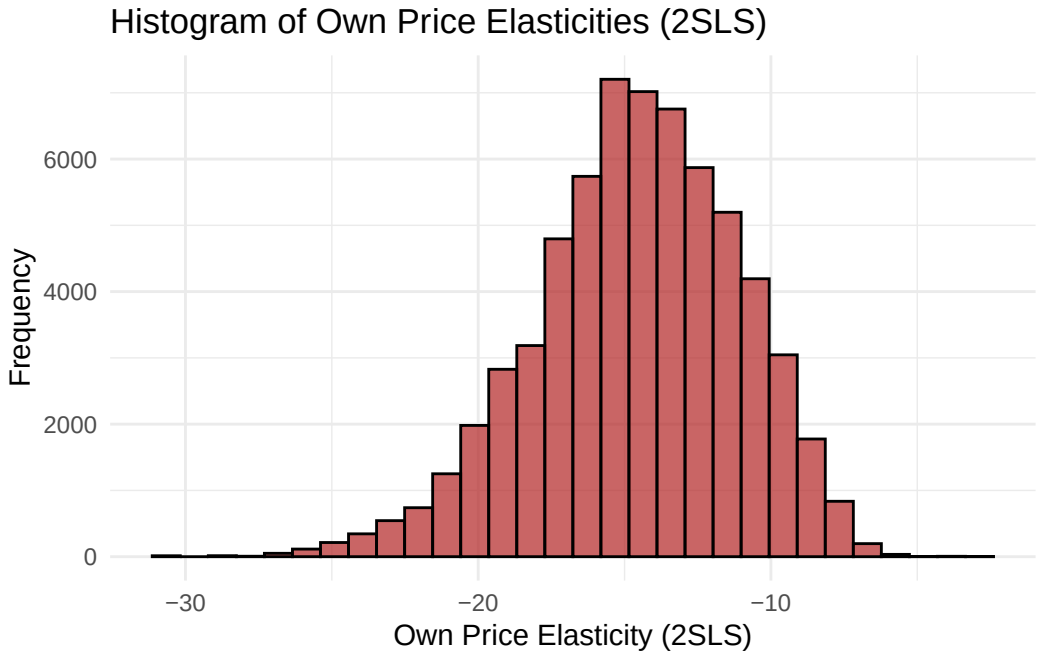
2.3

```r
# 2SLS with Price Instrument: Price*Quantity of Sugar

reg_data <- reg_data %>%
  mutate(
    z = sugars * sugar_price,
    z = log(z)

  )
stage2 <- feols(u ~ fiber + sugars + flavored + fortified |
                market_fe + manufacturer_fe + time_fe | p_jmt ~ z,
```

```
               data = reg_data)
summary(stage2)
```

```
TSLS estimation - Dep. Var.: u
                  Endo.    : p_jmt
                  Instr.   : z
Second stage: Dep. Var.: u
Observations: 63,966
Fixed-effects: market_fe: 642,  manufacturer_fe: 4,  time_fe: 16
Standard-errors: IID
            Estimate Std. Error   t value   Pr(>|t|)
fit_p_jmt -62.766445   7.898427  -7.94670 1.9468e-15 ***
fiber      -0.570755   0.066161  -8.62678  < 2.2e-16 ***
sugars     -0.120572   0.011575 -10.41695  < 2.2e-16 ***
flavored    0.179014   0.023118   7.74347 9.8166e-15 ***
fortified   0.377270   0.059710   6.31838 2.6607e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 2.49435     Adj. R2: 0.137296
                 Within R2: 0.025342
F-test (1st stage), p_jmt: stat =  54.680, p = 1.436e-13, on 1 and 63,301 DoF.
               Wu-Hausman: stat = 367.908, p < 2.2e-16  , on 1 and 63,300 DoF.
```

```
# Histogram of own price elasticities (2SLS)
reg_data <- reg_data %>%
  mutate(own_price_elasticity_2sls = coef(stage2)["fit_p_jmt"] * p_jmt * (1 - sj))
ggplot(reg_data, aes(x = own_price_elasticity_2sls)) +
  geom_histogram(bins = 30, fill = "firebrick", color = "black", alpha = 0.7) +
  labs(title = "Histogram of Own Price Elasticities (2SLS)",
       x = "Own Price Elasticity (2SLS)", y = "Frequency") +
  theme_minimal()
```




```
ggsave("q2-3-1.pdf", width = 10, height = 6, dpi = 300)
```

Using 2SLS results in estimates which are more negative than the OLS estimates, indicating that the OLS estimates were biased towards zero.

2.4

```
# Elasticity Matrix for Top 5 Brands by Sales
## Create average matrix by taking average of market level elaticity matrices

top_brands <- reg_data %>%
  group_by(brand) %>%
  summarise(total_sales = sum(quantity, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(total_sales)) %>%
  slice_head(n = 5) %>%
  pull(brand)
elasticity_matrices <- list()
for (mkt in unique(reg_data$market_id)) {
  mkt_data <- reg_data %>% filter(market_id == mkt, brand %in% top_brands)
  n <- nrow(mkt_data)
  if (n < 5) next  # Skip markets with fewer than 5 top brands
```

```r
  elas_matrix <- matrix(0, nrow = n, ncol = n)
  rownames(elas_matrix) <- mkt_data$brand
  colnames(elas_matrix) <- mkt_data$brand
  for (i in 1:n) {
    for (j in 1:n) {
      if (i == j) {
        elas_matrix[i, j] <- coef(stage2)["fit_p_jmt"] * mkt_data$p_jmt[i] * (1 - mkt_data$s
      } else {
        elas_matrix[i, j] <- -coef(stage2)["fit_p_jmt"] * mkt_data$p_jmt[j] * mkt_data$sj[i]
      }
    }
  }
  elasticity_matrices[[as.character(mkt)]] <- elas_matrix
}
# Average elasticity matrix
avg_elasticity_matrix <- Reduce("+", elasticity_matrices) / length(elasticity_matrices)
avg_elasticity_matrix
```

```
                            KELLOGGS FROSTED MINI WHEATS
KELLOGGS FROSTED MINI WHEATS                   -13.1423398
GENERAL MILLS CHEERIOS                           0.2615167
GENERAL MILLS HONEY NUT CHEER                    0.2654592
POST HONEY BUNCHES OF OATS                       0.2657008
KELLOGGS FROSTED FLAKES                          0.2576447
                            GENERAL MILLS CHEERIOS
KELLOGGS FROSTED MINI WHEATS              0.2638177
GENERAL MILLS CHEERIOS                  -13.2669948
GENERAL MILLS HONEY NUT CHEER             0.2671759
POST HONEY BUNCHES OF OATS                0.2681528
KELLOGGS FROSTED FLAKES                   0.2604532
                            GENERAL MILLS HONEY NUT CHEER
KELLOGGS FROSTED MINI WHEATS                     0.2650948
GENERAL MILLS CHEERIOS                           0.2639448
GENERAL MILLS HONEY NUT CHEER                  -13.2314072
POST HONEY BUNCHES OF OATS                       0.2691572
KELLOGGS FROSTED FLAKES                          0.2576187
                            POST HONEY BUNCHES OF OATS
KELLOGGS FROSTED MINI WHEATS                  0.2623082
GENERAL MILLS CHEERIOS                        0.2636535
GENERAL MILLS HONEY NUT CHEER                 0.2664261
POST HONEY BUNCHES OF OATS                  -13.2162577
KELLOGGS FROSTED FLAKES                       0.2599669
```

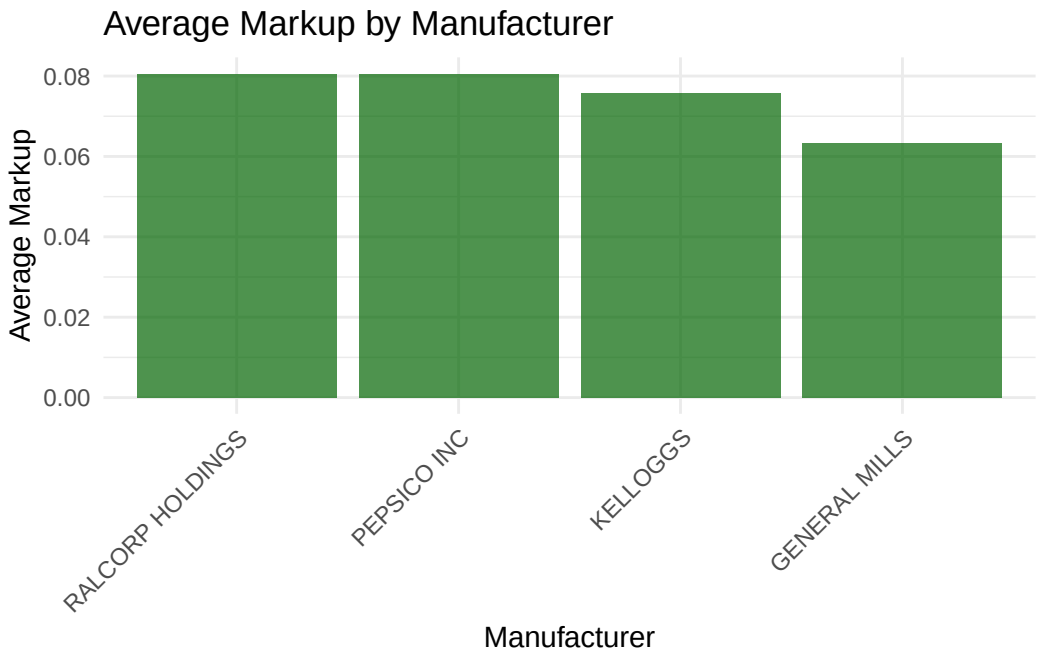
```
                              KELLOGGS FROSTED FLAKES
KELLOGGS FROSTED MINI WHEATS               0.2669329
GENERAL MILLS CHEERIOS                     0.2632467
GENERAL MILLS HONEY NUT CHEER              0.2654629
POST HONEY BUNCHES OF OATS                 0.2690599
KELLOGGS FROSTED FLAKES                  -13.2852126
```

2.5

```
# Calculaete markups for each manufacturer
reg_data <- reg_data %>%
  mutate(
    own_price_elasticity_2sls = coef(stage2)["fit_p_jmt"] * p_jmt * (1 - sj),
    markup = -1 / own_price_elasticity_2sls
  )
markup_summary <- reg_data %>%
  group_by(parent) %>%
  summarise(
    avg_markup = mean(markup, na.rm = TRUE),
    sd_markup = sd(markup, na.rm = TRUE),
    median_markup = median(markup, na.rm = TRUE),
    max_markup = max(markup, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(avg_markup))
markup_summary
```

```
# A tibble: 4 x 5
  parent           avg_markup sd_markup median_markup max_markup
  <chr>                 <dbl>     <dbl>         <dbl>      <dbl>
1 RALCORP HOLDINGS     0.0806    0.0225        0.0764      0.320
2 PEPSICO INC          0.0805    0.0179        0.0786      0.226
3 KELLOGGS             0.0757    0.0182        0.0727      0.182
4 GENERAL MILLS        0.0634    0.0127        0.0627      0.269
```

```
ggplot(markup_summary, aes(x = reorder(parent, -avg_markup), y = avg_markup)) +
  geom_bar(stat = "identity", fill = "darkgreen", alpha = 0.7) +
  labs(title = "Average Markup by Manufacturer",
       x = "Manufacturer", y = "Average Markup") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```
ggsave("q2-5-1.pdf", width = 10, height = 6, dpi = 300)
```

2.6

```
# Nested by segment - construct data and calculate conditional shares
reg_data <- reg_data %>%
  mutate(
    nest = as.factor(segment)
  ) %>%
  group_by(market_id, nest) %>%
  mutate(
    total_nest_sales = sum(quantity, na.rm = TRUE),
    share_con = if_else(total_nest_sales > 0, quantity / total_nest_sales, 0)
  ) %>%
  ungroup()
```

The estimating equation is now:

$$u_{jmt} = \alpha p_{jmt} + \beta \mathbb{X}_{jmt} + \sigma_g s_{j|g} + \xi_{jmt}$$

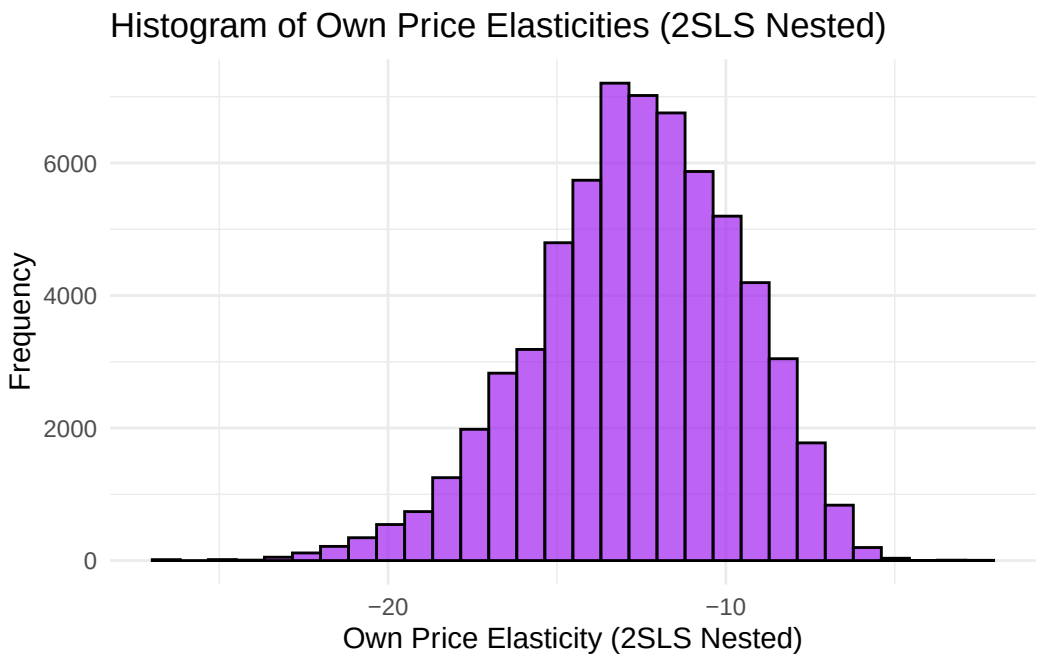
where $s_{j|g}$ is the conditional share of product j within its nest g.

2.7

```
#2SLS with Price Instrument: Price*Quantity of Sugar and Conditional Share Instrument total p

reg_data <- reg_data %>%
  group_by(market_id) %>%
  mutate(
    z_cond = log(n_distinct(brand)),
    sjg = log(share_con),
    z = sugars * sugar_price,
    z = log(z)
  )
s2_nest <- feols(u ~ fiber + sugars + flavored + fortified |
                   market_fe + manufacturer_fe + time_fe | p_jmt + sjg ~ z + z_cond,
                 data = reg_data)
summary(s2_nest)
```

```
TSLS estimation - Dep. Var.: u
                  Endo.    : p_jmt, sjg
                  Instr.   : z, z_cond
Second stage: Dep. Var.: u
Observations: 63,966
Fixed-effects: market_fe: 642,  manufacturer_fe: 4,  time_fe: 16
Standard-errors: IID
            Estimate Std. Error  t value   Pr(>|t|)
fit_p_jmt -54.396707   6.652299 -8.17713 2.9602e-16 ***
fit_sjg     0.129922   0.078950  1.64563 9.9845e-02 .
fiber      -0.486850   0.059327 -8.20618 2.3258e-16 ***
sugars     -0.100596   0.012322 -8.16377 3.3063e-16 ***
flavored    0.174628   0.020285  8.60859  < 2.2e-16 ***
fortified   0.300395   0.055403  5.42196 5.9163e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 2.15952     Adj. R2: 0.137644
                Within R2: 0.025751
F-test (1st stage), p_jmt: stat =  32.803, p = 5.77e-15, on 2 and 63,300 DoF.
F-test (1st stage), sjg  : stat = 472.922, p < 2.2e-16 , on 2 and 63,300 DoF.
              Wu-Hausman: stat = 942.470, p < 2.2e-16 , on 2 and 63,298 DoF.
```

```r
# Histogram of own price elasticities (2SLS Nested)
reg_data <- reg_data %>%
  mutate(own_price_elasticity_2sls_nest = coef(s2_nest)["fit_p_jmt"] * p_jmt * (1 - sj),
  sgj = log(coef(s2_nest)["fit_share_con"]))
ggplot(reg_data, aes(x = own_price_elasticity_2sls_nest)) +
  geom_histogram(bins = 30, fill = "purple", color = "black", alpha = 0.7) +
  labs(title = "Histogram of Own Price Elasticities (2SLS Nested)",
       x = "Own Price Elasticity (2SLS Nested)", y = "Frequency") +
  theme_minimal()
```




```r
ggsave("q2-7-1.pdf", width = 10, height = 6, dpi = 300)
```

2.8

```r
# Elasticity Matrix for Top 5 Brands by Sales (Nested)
top_nest_brands <- reg_data %>%
  group_by(brand) %>%
  summarise(total_sales = sum(quantity, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(total_sales)) %>%
  slice_head(n = 5) %>%
  pull(brand)
```

```r
elasticity_nest_matrices <- list()
for (mkt in unique(reg_data$market_id)) {
  mkt_data <- reg_data %>% filter(market_id == mkt, brand %in% top_nest_brands)
  n <- nrow(mkt_data)
  if (n < 5) next  # Skip markets with fewer than 5 top brands
  elas_matrix_nest <- matrix(0, nrow = n, ncol = n)
  rownames(elas_matrix_nest) <- mkt_data$brand
  colnames(elas_matrix_nest) <- mkt_data$brand
  for (i in 1:n) {
    for (j in 1:n) {
      if (i == j) {
        elas_matrix_nest[i, j] <- coef(s2_nest)["fit_p_jmt"] *
          mkt_data$p_jmt[i] * (1 / (1 - coef(s2_nest)["fit_sjg"])) *
          ((1 - coef(s2_nest)["fit_sjg"] * mkt_data$sjg[i]) - (1
            - coef(s2_nest)["fit_sjg"]) * mkt_data$sj[i])
      } else if (mkt_data$nest[i] == mkt_data$nest[j]) {
        elas_matrix_nest[i, j] <- -coef(s2_nest)["fit_p_jmt"] *
          mkt_data$p_jmt[i] * (1 / (1 - coef(s2_nest)["fit_sjg"])) *
          ((1 - coef(s2_nest)["fit_sjg"] * mkt_data$sjg[i]) - (1
            - coef(s2_nest)["fit_sjg"]) * mkt_data$sj[i])
      } else {
        elas_matrix_nest[i, j] <- coef(s2_nest)["fit_p_jmt"] *
          mkt_data$p_jmt[i] * (1 / (1 - coef(s2_nest)["fit_sjg"])) *
          ((1 - coef(s2_nest)["fit_sjg"] * mkt_data$sjg[i]) - (1 -
            coef(s2_nest)["fit_sjg"]) * mkt_data$sj[i])

      }
    }
  }
  elasticity_nest_matrices[[as.character(mkt)]] <- elas_matrix_nest
}
# Average elasticity matrix (Nested)
avg_elasticity_nest_matrix <- Reduce("+", elasticity_nest_matrices) /
  length(elasticity_nest_matrices)
avg_elasticity_nest_matrix
```

```
                            KELLOGGS FROSTED MINI WHEATS
KELLOGGS FROSTED MINI WHEATS                   -16.726258
GENERAL MILLS CHEERIOS                           3.934947
GENERAL MILLS HONEY NUT CHEER                    3.884717
POST HONEY BUNCHES OF OATS                       4.220221
KELLOGGS FROSTED FLAKES                          3.412595
```

```
                            GENERAL MILLS CHEERIOS
KELLOGGS FROSTED MINI WHEATS               3.804315
GENERAL MILLS CHEERIOS                   -16.887474
GENERAL MILLS HONEY NUT CHEER              4.515091
POST HONEY BUNCHES OF OATS                 4.865219
KELLOGGS FROSTED FLAKES                    4.027965
                            GENERAL MILLS HONEY NUT CHEER
KELLOGGS FROSTED MINI WHEATS                     3.769583
GENERAL MILLS CHEERIOS                           4.375811
GENERAL MILLS HONEY NUT CHEER                  -16.825855
POST HONEY BUNCHES OF OATS                       4.791045
KELLOGGS FROSTED FLAKES                          3.976238
                            POST HONEY BUNCHES OF OATS
KELLOGGS FROSTED MINI WHEATS                   4.271827
GENERAL MILLS CHEERIOS                         5.006285
GENERAL MILLS HONEY NUT CHEER                  4.941390
POST HONEY BUNCHES OF OATS                   -16.782152
KELLOGGS FROSTED FLAKES                        4.387227
                            KELLOGGS FROSTED FLAKES
KELLOGGS FROSTED MINI WHEATS                3.286264
GENERAL MILLS CHEERIOS                      3.794710
GENERAL MILLS HONEY NUT CHEER               3.807976
POST HONEY BUNCHES OF OATS                  4.280453
KELLOGGS FROSTED FLAKES                   -16.886185
```

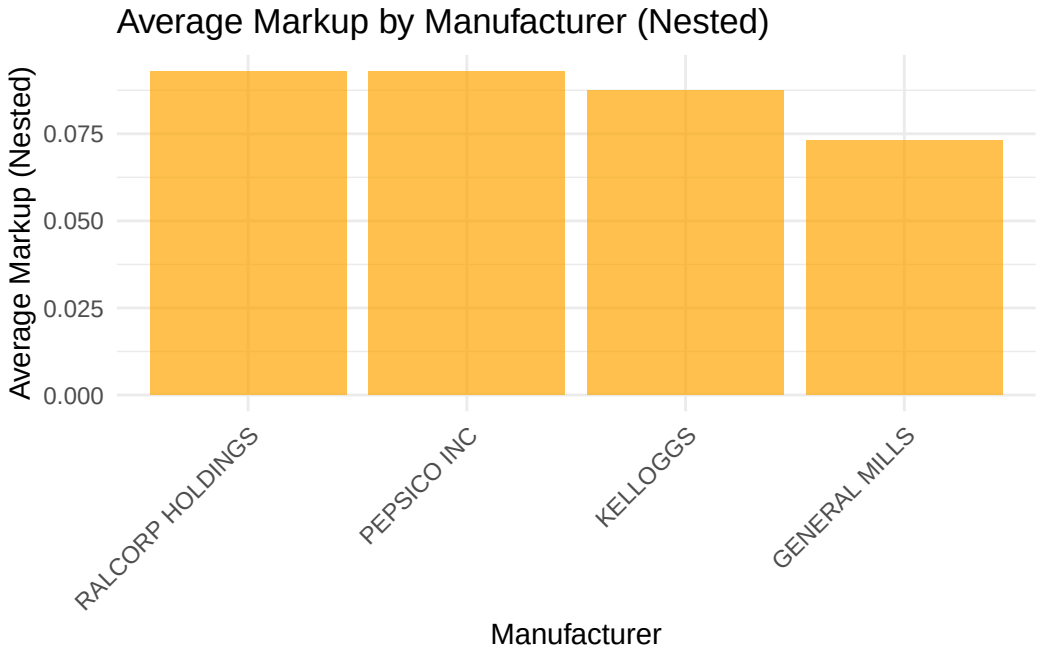
2.9

```r
# Calculate average markups for each manufacturer (Nested)
reg_data <- reg_data %>%
  mutate(
    ope_nest = coef(s2_nest)["fit_p_jmt"] * p_jmt * (1 - sj),
    markup_nest = -1 / ope_nest
  )
markup_nest_summary <- reg_data %>%
  group_by(parent) %>%
  summarise(
    avg_markup_nest = mean(markup_nest, na.rm = TRUE),
    sd_markup_nest = sd(markup_nest, na.rm = TRUE),
    median_markup_nest = median(markup_nest, na.rm = TRUE),
    max_markup_nest = max(markup_nest, na.rm = TRUE),
    .groups = "drop"
```

```
  ) %>%
  arrange(desc(avg_markup_nest))
markup_nest_summary
```

```
# A tibble: 4 x 5
  parent       avg_markup_nest sd_markup_nest median_markup_nest max_markup_nest
  <chr>                  <dbl>          <dbl>              <dbl>           <dbl>
1 RALCORP HOL~           0.0930         0.0259             0.0881           0.370
2 PEPSICO INC            0.0929         0.0206             0.0907           0.261
3 KELLOGGS               0.0873         0.0210             0.0839           0.210
4 GENERAL MIL~           0.0731         0.0146             0.0723           0.311
```

```
ggplot(markup_nest_summary, aes(x = reorder(parent, -avg_markup_nest),
                                y = avg_markup_nest)) +
  geom_bar(stat = "identity", fill = "orange", alpha = 0.7) +
  labs(title = "Average Markup by Manufacturer (Nested)",
       x = "Manufacturer", y = "Average Markup (Nested)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```
ggsave("q2-9-1.pdf", width = 10, height = 6, dpi = 300)
```

2.10

```
# Cross - market diversion ratios between cheerios and corn flakes/froot loops
brands <- c("GENERAL MILLS CHEERIOS", "KELLOGGS CORN FLAKES", "KELLOGGS FROOT LOOPS")

row_stats <- reg_data %>%
  filter(brand %in% brands) %>%
  mutate(sj = pmin(sj, 0.999999)) %>%
  group_by(brand) %>%
  summarise(row_value = mean(-own_price_elasticity_2sls_nest * (sj / (1 - sj)), na.rm = TRUE
            .groups = "drop")

diversion_matrix <- matrix(NA_real_, 3, 3, dimnames = list(brands, brands))
row_vals <- setNames(row_stats$row_value, row_stats$brand)

for (b in brands) {
  if (!is.na(row_vals[b])) diversion_matrix[b, setdiff(brands, b)] <- row_vals[b]
}

diversion_matrix
```

```
                       GENERAL MILLS CHEERIOS KELLOGGS CORN FLAKES
GENERAL MILLS CHEERIOS                     NA           0.23218833
KELLOGGS CORN FLAKES              0.10098464                   NA
KELLOGGS FROOT LOOPS              0.08138226           0.08138226
                       KELLOGGS FROOT LOOPS
GENERAL MILLS CHEERIOS            0.2321883
KELLOGGS CORN FLAKES              0.1009846
KELLOGGS FROOT LOOPS                     NA
```

2.11

```
# Find average markup for a hypothetical merger between General Mills and Kelloggs
merged_markup <- reg_data %>%
  mutate(
    parent_merged = if_else(parent %in% c("GENERAL MILLS", "KELLOGGS"), "GM_K", parent)
  ) %>%
  group_by(parent_merged) %>%
  summarise(
```

```
    avg_markup_merged = mean(markup_nest, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(avg_markup_merged))
merged_markup
```

```
# A tibble: 3 x 2
  parent_merged    avg_markup_merged
  <chr>                        <dbl>
1 RALCORP HOLDINGS            0.0930
2 PEPSICO INC                 0.0929
3 GM_K                        0.0799
```

```
# The average markup for the merged entity "GM_K" is:
merged_markup %>% filter(parent_merged == "GM_K") %>% pull(avg_markup_merged)
```

```
[1] 0.07994269
```

```
# This is higher than the individual average markups of General Mills and Kelloggs,
# indicating that the merger could lead to increased market power and higher markups
```

3.1

```
import pandas as pd
import numpy as np
import pyblp

# Set random seed for reproducibility
np.random.seed(42)

# ============================================================================
# LOAD AND PREPARE DATA
# ============================================================================

print("Loading data...")
df = pd.read_csv("~/SchoolWork/Y2S1/IO/PS1/iri.csv", sep='\t')

# Load demographics
income_df = pd.read_csv("simulated_agents_income.csv", sep='\t')
nchild_df = pd.read_csv("simulated_agents_nchild.csv", sep='\t')
```

```
# Create market ID
df['market_ids'] = df['store_id'].astype(str) + '_' + df['week_id'].astype(str)

# Calculate shares
df['shares'] = df['quantity'] / df['M']

# PyBLP requires 'prices' (plural) column
df['prices'] = df['price']

# PyBLP requires 'firm_ids' for supply-side or when using X3
df['firm_ids'] = df['parent']  # Parent company is the firm

# Create product IDs (must be unique within market)
df['product_ids'] = df['brand'] + '_' + df['parent']

# Take subset for testing
test_markets = df['market_ids'].unique()[:500]
df_test = df[df['market_ids'].isin(test_markets)].copy().reset_index(drop=True)

print(f"Dataset: {len(df_test)} obs, {df_test['market_ids'].nunique()} markets")
print(f"Share range: [{df_test['shares'].min():.4f}, {df_test['shares'].max():.4f}]")

# ============================================================================
# CREATE INSTRUMENTS
# ============================================================================

print("\nCreating instruments...")

# BLP instruments - competitor characteristics
for char in ['sugars', 'fiber']:
    df_test[f'blp_{char}'] = 0.0

    for market in df_test['market_ids'].unique():
        market_mask = df_test['market_ids'] == market
        market_data = df_test[market_mask]
        n_products = len(market_data)

        if n_products <= 1:
            continue

        for idx in market_data.index:
            competitors = df_test[(df_test['market_ids'] == market) & (df_test.index != idx)]
```

```
            if len(competitors) > 0:
                comp_sum = competitors[char].sum()
                own_val = df_test.loc[idx, char]
                # BLP formula: (mean_competitors - own)^2
                df_test.loc[idx, f'blp_{char}'] = ((comp_sum / len(competitors)) - own_val) *

# Cost shifter: sugar price × sugar content
df_test['demand_instruments0'] = np.log(df_test['sugars'] * df_test['sugar_price'] + 0.01)
df_test['demand_instruments1'] = df_test['blp_sugars']
df_test['demand_instruments2'] = df_test['blp_fiber']

print("Instruments created")

# ============================================================================
# PREPARE AGENT DATA (Demographics)
# ============================================================================

print("\nPreparing agent data...")

n_draws = 50  # Number of simulation draws per market
income_cols = [col for col in income_df.columns if col.startswith('income')][:n_draws]
nchild_cols = [col for col in nchild_df.columns if col.startswith('nchild')][:n_draws]

agent_data_list = []

for market in df_test['market_ids'].unique():
    market_df = df_test[df_test['market_ids'] == market].iloc[0:1]

    # Get puma and year for this market
    puma = market_df['puma'].values[0]
    year = market_df['year'].values[0]

    # Get demographic draws for this puma/year
    income_row = income_df[(income_df['puma'] == puma) & (income_df['year'] == year)]
    nchild_row = nchild_df[(nchild_df['puma'] == puma) & (nchild_df['year'] == year)]

    if len(income_row) > 0 and len(nchild_row) > 0:
        incomes = income_row[income_cols].values[0] / 10000  # Normalize
        nchilds = nchild_row[nchild_cols].values[0]

        # Create agent data for this market
        for i in range(n_draws):
            agent_data_list.append({
```

```
                'market_ids': market,
                'weights': 1.0 / n_draws,
                'income': incomes[i],
                'nchild': nchilds[i],
                'nodes0': np.random.randn(),  # For sugar RC
                'nodes1': np.random.randn()   # For fiber RC
            })

agent_data = pd.DataFrame(agent_data_list)
print(f"Agent data: {len(agent_data)} agents across {agent_data['market_ids'].nunique()} mar

# ============================================================================
# PYBLP FORMULATIONS
# ============================================================================

print("\n" + "="*70)
print("SETTING UP PYBLP PROBLEM")
print("="*70)

# X1: Linear parameters (including prices which will be instrumented)
product_formulations = (
    pyblp.Formulation('0 + prices + flavored + fortified + fiber + sugars'),
    # X2: Random coefficients on sugar and fiber
    pyblp.Formulation('0 + sugars + fiber'),
    # X3: Demographic interactions (sugar×income, fiber×income)
    pyblp.Formulation('0 + blp_sugars + blp_fiber')
)

# Agent formulation for demographics
agent_formulation = pyblp.Formulation('0 + income + nchild')

# Create problem
problem = pyblp.Problem(
    product_formulations=product_formulations,
    product_data=df_test,
    agent_formulation=agent_formulation,
    agent_data=agent_data
)

print(problem)

# ============================================================================
# INITIAL VALUES
```

```
# ============================================================================

print("\n" + "="*70)
print("SETTING INITIAL PARAMETERS")
print("="*70)

# Sigma (random coefficient std deviations) for [sugars, fiber]
initial_sigma = np.diag([0.1, 0.1])

# Pi (demographic interactions) for [sugars×income, sugars×nchild; fiber×income, fiber×nchild
initial_pi = np.array([
    [0.01, 0.01],  # sugars interactions
    [0.01, 0.01]   # fiber interactions
])

# Beta (linear parameters) - REQUIRED when firm_ids is present
# Order: [prices, flavored, fortified, fiber, sugars]
initial_beta = np.array([
    [-3.0],   # price coefficient (negative)
    [0.01],    # flavored
    [0.01],    # fortified
    [0.1],    # fiber (positive preference)
    [0.1]    # sugars (negative preference for too much sugar)
])

print(f"Initial Sigma:\n{initial_sigma}")
print(f"Initial Pi:\n{initial_pi}")
print(f"Initial Beta:\n{initial_beta.flatten()}")

# ============================================================================
# ESTIMATION
# ============================================================================

print("\n" + "="*70)
print("STARTING ESTIMATION (this may take several minutes...)")
print("="*70)

# Solve the problem
results = problem.solve(
    sigma=initial_sigma,
    pi=initial_pi,
    beta=initial_beta,  # ADD initial beta values
    optimization=pyblp.Optimization('l-bfgs-b', {
```

```
        'maxiter': 1000,
        'gtol': 1e-5
    }),
    iteration=pyblp.Iteration('squarem', {
        'atol': 1e-12,
        'max_evaluations': 1000
    })
)

# ============================================================================
# RESULTS
# ============================================================================

print("\n" + "="*70)
print("ESTIMATION RESULTS")
print("="*70)
print(results)

# Extract and display parameters
print("\n" + "="*70)
print("PARAMETER ESTIMATES")
print("="*70)

print("\n1. Linear Parameters ( ):")
beta_df = pd.DataFrame({
    'Parameter': ['prices', 'flavored', 'fortified', 'fiber', 'sugars'],
    'Estimate': results.beta.flatten(),
    'Std Error': results.beta_se.flatten()
})
print(beta_df.to_string(index=False))

print("\n2. Random Coefficient Std Deviations (Σ):")
sigma_df = pd.DataFrame({
    'Parameter': [' _sugars', ' _fiber'],
    'Estimate': np.diag(results.sigma),
    'Std Error': np.diag(results.sigma_se) if results.sigma_se is not None else [np.nan, np.n
})
print(sigma_df.to_string(index=False))

print("\n3. Demographic Interactions (Π):")
pi_df = pd.DataFrame(
    results.pi,
    index=['sugars', 'fiber'],
```

```
    columns=['income', 'nchild']
)
print("Estimates:")
print(pi_df)

if results.pi_se is not None:
    pi_se_df = pd.DataFrame(
        results.pi_se,
        index=['sugars', 'fiber'],
        columns=['income', 'nchild']
    )
    print("\nStandard Errors:")
    print(pi_se_df)

# Diagnostics
print("\n" + "="*70)
print("DIAGNOSTICS")
print("="*70)

print(f"GMM Objective Value: {float(results.objective):.6e}")
print(f"Optimization Status: {float(results.optimization_time):.6e}")


print("\n" + "="*70)
print("ESTIMATION COMPLETE")
print("="*70)

# Save results if desired
# results_df = pd.DataFrame({
#     'beta': results.beta.flatten(),
#     'sigma': np.diag(results.sigma)
# })
# results_df.to_csv('blp_results.csv', index=False)

# Create histogram of own price elasticities
elasticities = results.compute_elasticities()
own_price_elasticities = np.diag(elasticities)
import matplotlib.pyplot as pyplot
pyplot.hist(own_price_elasticities, bins=30, edgecolor='k')
pyplot.title('Histogram of Own Price Elasticities')
pyplot.xlabel('Own Price Elasticity')
pyplot.ylabel('Frequency')
pyplot.grid(True)
```

```
pyplot.show()
print("\nHistogram of own price elasticities displayed.")

# Construct elasticity matrix for top 5 brands in terms of sales (own and cross-price elasti

top_brands = df_test.groupby('brand')['quantity'].sum().nlargest(5).index
top_brand_data = df_test[df_test['brand'].isin(top_brands)].copy().reset_index(drop=True)
top_brand_problem = pyblp.Problem(
    product_formulations=product_formulations,
    product_data=top_brand_data,
    agent_formulation=agent_formulation,
    agent_data=agent_data
)
top_brand_problem.solve(
    sigma=results.sigma,
    pi=results.pi,
    beta=results.beta
    top_brand_elasticities = top_brand_problem.compute_elasticities()
)

print("\nTop 5 Brands Elasticity Matrix (Own and Cross-Price Elasticities):")
elasticity_matrix = pd.DataFrame(
    top_brand_elasticities,
    index=top_brand_data['brand'],
    columns=top_brand_data['brand']
)
print(elasticity_matrix)
```